

DESIGN AND DEVELOPMENT OF A LIGHTWEIGHT RUNTIME QUALITY ASSESSMENT FRAMEWORK FOR CLOUD-NATIVE MICROSERVICE APPLICATIONS

SEZG628T: Dissertation

by

Ganesh Ambadas Palve

2024TM93686

Dissertation work carried out at

Bajaj Finserv, Pune

Submitted in partial fulfilment of M.Tech. Software Engineering
degree programme

Under the Supervision of

Ajay Singh

Bajaj Finserv, Pune

**BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE
PILANI (RAJASTHAN)**

September 2026

ABSTRACT

Microservice applications are monitored with tools such as Prometheus and Grafana, which collect many runtime metrics — response time, error rate, availability, CPU and memory usage — but present each of them separately. Operators must read several dashboards and decide for themselves whether the system as a whole is healthy, whether it is getting better or worse, and which service is responsible. This dissertation designs and develops a lightweight **Runtime Quality Assessment Framework (QAF)** that turns these metrics into one interpretable **Quality Index (QI, 0–100)** for every service and for the whole system, classifies it as Excellent, Good, Fair, Poor or Critical, identifies the service contributing most to quality loss, and recommends corrective actions.

Up to the mid-semester point, we defined a quality model with five dimensions (Performance, Reliability, Availability, Resource Efficiency and Scalability) mapped to seven Prometheus metrics, with threshold-based normalisation and dimension weights derived by the Analytic Hierarchy Process (consistency ratio 0.012); the Entropy Weight Method and a hybrid of the two are also implemented. We built the complete framework in Python (metric collection, noise and missing-data handling, scoring, weighting, classification, degradation attribution and a rule-based recommendation engine) together with a four-service e-commerce prototype (User, Product, Order, Payment) in Node.js with fault injection, Prometheus, Grafana and k6 load tests, all running on Docker Compose. The framework is covered by 17 automated tests.

We then ran nine preliminary experiments, once each, covering low, medium, peak and spike load and five injected failures. The system QI ranged from 98.5 under medium load to 49.8 when the payment service was stopped, and in every failure scenario the framework pointed to the affected service. In three of the nine scenarios, conventional static alert thresholds would not have fired while the QI showed a clear drop. The experiments also exposed limitations — a compensatory weighted average can hide a single critical metric, and errors propagated from a dependency are partly blamed on the caller — which are planned for the second half of the semester, together with repeated runs, comparison of weighting methods, a structured comparison with dashboard-based monitoring and a Kubernetes deployment.

Signature of the Student

Name: Ganesh Ambadas Palve

Date:

Place: Pune

Signature of the Supervisor

Name: Ajay Singh

Date:

Place: Pune

CONTENTS

1. INTRODUCTION	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Objectives	5
1.4 Scope of Work	6
1.5 Progress at Mid-Semester	6
2. LITERATURE REVIEW	7
2.1 Microservices and Observability	7
2.2 Software Quality Models	7
2.3 Composite Indices and Multi-Criteria Weighting	7
2.4 Summary of Reviewed Literature	8
2.5 Research Gap	8
3. FRAMEWORK DESIGN	9
3.1 Overall Architecture	9
3.2 Framework Modules	9
3.3 Quality Model	10
3.4 Metric Normalisation	10
3.5 Weight Assignment	11
3.6 Quality Index and Classification	12
3.7 System Quality Index and Attribution of Quality Loss	12
3.8 Recommendation Engine	12
4. IMPLEMENTATION	14
4.1 Technology Stack	14
4.2 Microservice Prototype	14
4.3 Monitoring Setup	15
4.4 Quality Assessment Engine	15
4.5 Testing	15
4.6 Source Code Repository	15
5. EXPERIMENTAL SETUP AND PRELIMINARY RESULTS	17
5.1 Experimental Setup	17
5.2 System Quality Index per Scenario	17
5.3 Service-Level Quality Index	18
5.4 Quality Index over Time	19
5.5 Dimension Scores and Recommendations	19
5.6 Preliminary Comparison with Threshold-Based Monitoring	20
5.7 Observations and Limitations	21
6. FUTURE PLAN	23
7. REFERENCES	25
8. ABBREVIATIONS	27

LIST OF FIGURES

Figure 1: Architecture of the Runtime Quality Assessment Framework and prototype

Figure 2: Dimension weights derived by AHP

Figure 3: Mean system Quality Index per scenario with quality-level bands

Figure 4: Mean service Quality Index per scenario

Figure 5: Service and system QI over time for E3, E5 and E9

Figure 6: Dimension scores of the affected service in selected scenarios (E6 performance = 0)

LIST OF TABLES

Table 1: Status of objectives at mid-semester

Table 2: Summary of reviewed literature

Table 3: Framework modules

Table 4: Quality dimensions, metrics and normalisation thresholds

Table 5: AHP pairwise comparison matrix of the quality dimensions

Table 6: Quality levels

Table 7: Recommendation rules

Table 8: Technology stack of the prototype

Table 9: Framework command-line interface

Table 10: Experiment scenarios

Table 11: System QI and lowest-scoring service per scenario

Table 12: Static alert rules compared with the framework (degraded scenarios)

Table 13: Plan of work and status

Table 14: Abbreviations

1. INTRODUCTION

1.1 Background

Organisations are moving large applications from monoliths to cloud-native microservices so that teams can build, deploy and scale each service on its own. The price of this flexibility is operational complexity: a single user request may pass through several services, each running in its own container, and each exposing its own stream of metrics. Monitoring tools such as Prometheus collect these metrics — latency, request rate, error rate, availability, CPU and memory — and Grafana draws them as charts and dashboards.

These tools are very good at collecting and displaying data, but they leave the *interpretation* to people. To answer a simple question such as “is the system healthy right now?”, an engineer must look at several panels per service, remember what a normal value is for each metric, and mentally combine them. The same difficulty appears when comparing two releases or two deployments: there is no single number that says which one is better.

1.2 Problem Statement

Existing monitoring platforms collect metrics, display dashboards and raise alerts on individual thresholds. They do not directly answer the following questions:

- Is the system healthy **overall**, taking all quality attributes into account together?
- Is service quality improving or degrading over time or across deployments?
- **Which microservice** contributes most to the loss of quality?
- Can two deployments be compared objectively with one number?

This dissertation addresses these questions by designing a framework that combines multiple runtime metrics into a single, explainable Quality Index.

1.3 Objectives

The objectives, as approved in the dissertation outline, are:

1. Study the runtime quality attributes relevant to cloud-native microservice applications.
2. Identify measurable runtime metrics collected by standard monitoring tools such as Prometheus.
3. Design a lightweight Quality Assessment Framework comprising metric collection, preprocessing, quality analysis and indexing layers.
4. Develop a Quality Index model that aggregates Performance, Reliability, Availability, Resource Efficiency and Scalability scores using a defined weighting scheme.

5. Implement the framework on a representative microservice prototype (User, Product, Order and Payment services).
6. Validate the framework experimentally under low, medium, peak and failure-injected workloads.
7. Compare the results with traditional dashboard-based monitoring.

1.4 Scope of Work

The research contribution is the **framework** — the quality model, the scoring and weighting method, the classification and the attribution of quality loss — and its experimental validation. The microservice application is a proof-of-concept used to generate realistic runtime behaviour; it is deliberately small and runs on a single machine. A production-grade monitoring product, multi-cluster deployment and security auditing are outside the scope of this work.

1.5 Progress at Mid-Semester

Table 1 summarises the status of each objective at the time of this report.

Table 1: Status of objectives at mid-semester

#	Objective	Status	Evidence in this report
1	Study runtime quality attributes	Completed	Chapter 2
2	Identify measurable metrics	Completed	Section 3.3, Table 4
3	Design the framework	Completed	Chapter 3
4	Quality Index model and weighting	Completed	Sections 3.4–3.7
5	Implement on a prototype	Completed	Chapter 4
6	Experimental validation	In progress	Chapter 5 (one run per scenario)
7	Comparison with dashboard monitoring	In progress	Section 5.6 (preliminary)

2. LITERATURE REVIEW

2.1 Microservices and Observability

Microservices structure an application as a set of small services that communicate over the network and can be deployed independently [5], [6]. Surveys of practitioners report that this independence comes with new difficulties in operating and monitoring the resulting distributed system [12]. Observability practice has converged on a few compact sets of signals. Google’s Site Reliability Engineering book proposes the **four golden signals** — latency, traffic, errors and saturation — as the minimum to monitor for any user-facing service [4]. Gregg’s **USE method** looks at utilisation, saturation and errors of every resource [7], and the **RED method** (rate, errors, duration) adapts this to request-driven microservices [11]. Prometheus [8] has become the common open-source way to collect such metrics from containers.

These methods tell engineers *what to measure*, and they are the basis for the metrics chosen in this work. They do not, however, describe how to *combine* the signals into an overall judgement of quality, which is left to dashboards and human interpretation.

2.2 Software Quality Models

ISO/IEC 25010 defines a product quality model with characteristics such as performance efficiency, reliability (including availability and fault tolerance) and maintainability [1]. The five dimensions used in this framework follow this model for the characteristics that can be observed at runtime: performance (time behaviour), reliability (fault tolerance), availability, resource efficiency (resource utilisation) and scalability (capacity). Many studies of microservice quality focus on design-time properties such as coupling, cohesion and service granularity [13], [15], [16], while systematic reviews note that runtime quality assessment of microservices remains an open research area [14].

2.3 Composite Indices and Multi-Criteria Weighting

A composite index combines several indicators into one number. The OECD/JRC handbook describes the standard steps — selecting indicators, normalising them, weighting and aggregating — and warns that linear (weighted-average) aggregation is **compensatory**: a very poor value on one indicator can be hidden by good values on the others [10]. This point turned out to be important in our experiments (Section 5.7).

Weights can be assigned subjectively or objectively. The **Analytic Hierarchy Process (AHP)** derives weights from pairwise comparisons made by experts and checks that these judgements are consistent through a consistency ratio [2]. The **Entropy Weight Method (EWM)**, based on Shannon’s information entropy [3], derives weights from the data itself: indicators that vary more across the observations carry more information and receive higher weight [17]. Recent work on software quality in agile projects normalises code, process and team indicators, weights them with an entropy-based method and combines them into a single composite index, the Overall Software Quality Index [18]; this dissertation applies a similar idea to the runtime metrics of microservices, combining subjective (AHP) and objective (entropy) weighting.

2.4 Summary of Reviewed Literature

Table 2: Summary of reviewed literature

Ref.	Focus	Relevance to this work
[4], [7], [11]	Golden signals, USE and RED methods	Basis for choosing latency, errors, traffic and saturation metrics
[1]	ISO/IEC 25010 quality model	Defines the quality characteristics mapped to the five dimensions
[13], [15], [16]	Design-time microservice metrics	Complementary; they do not use runtime behaviour
[14]	Review of microservice quality assessment	Identifies runtime quality assessment as a gap
[2], [3], [17]	AHP and Entropy Weight Method	Weighting methods implemented in the framework
[10], [18]	Composite indicators; entropy-weighted software quality index	Normalisation and aggregation method; compensability issue

2.5 Research Gap

The reviewed work either (a) defines which runtime signals to monitor without combining them, (b) assesses microservice quality from the design rather than from runtime behaviour, or (c) builds composite quality indices in other domains. We found little work that combines runtime metrics of a running microservice system into a **single, weighted, explainable Quality Index** that also **attributes quality loss to individual services** and **suggests corrective actions**. This is the gap addressed by the proposed framework. As advised by the supervisor, the literature survey will be extended in the second half with more recent peer-reviewed work on QoS-based composite indices.

3. FRAMEWORK DESIGN

3.1 Overall Architecture

Figure 1 shows the architecture of the framework and the prototype used to evaluate it. The microservices expose their metrics on a /metrics endpoint; Prometheus scrapes them every five seconds. The framework queries Prometheus through its HTTP API, runs the assessment pipeline, and publishes the results in three ways: a console report, a CSV log for experiments, and a set of Prometheus metrics that Grafana displays on a dashboard. The framework does not need any change to the services beyond standard Prometheus instrumentation, which keeps it **lightweight and tool-agnostic**.

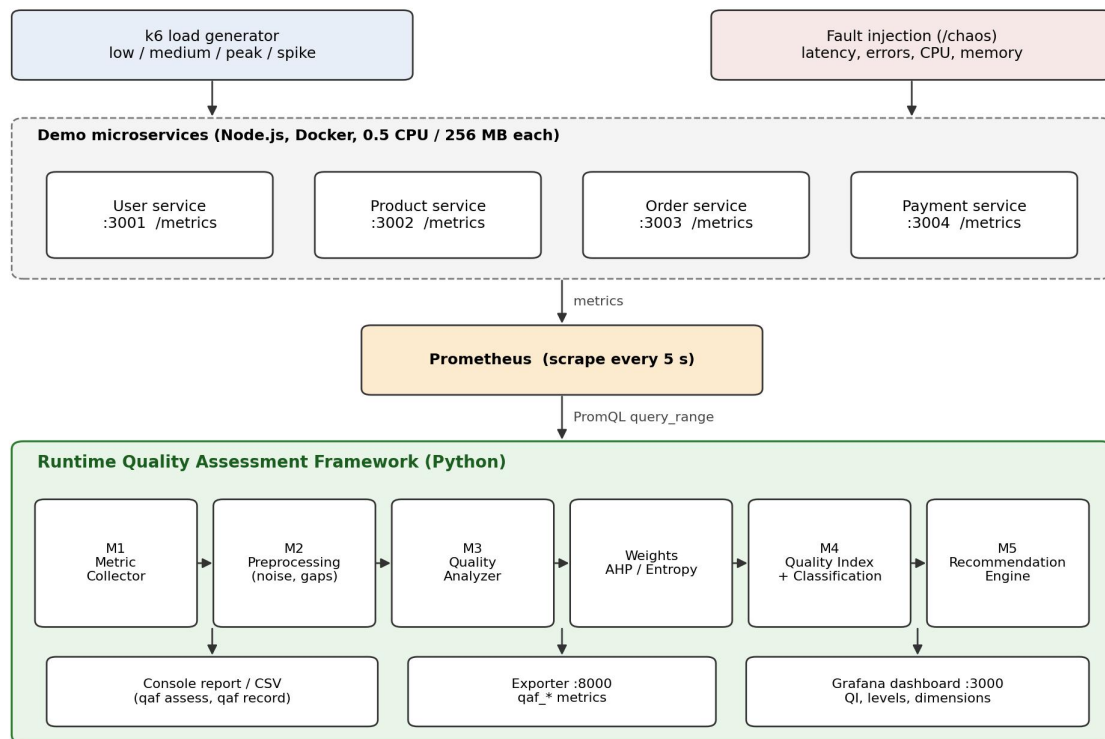


Figure 1: Architecture of the Runtime Quality Assessment Framework and prototype

3.2 Framework Modules

Table 3: Framework modules

Module	Responsibility	Main technique
M1 Metric Collector	Fetch raw metrics per service for the last 2 minutes	PromQL range queries (rate, histogram_quantile)
M2 Metric Preprocessing	Remove noise, handle missing data, aggregate a window into one value; detect services that are down	IQR outlier filter, median aggregation

Module	Responsibility	Main technique
M3 Quality Analyzer	Normalise metrics to 0–100 and combine them into five dimension scores	Threshold-based min–max normalisation
Weight Assignment	Assign weights to the five dimensions	AHP, Entropy Weight Method, hybrid or fixed
M4 QI Calculator and Classifier	Compute service and system QI, quality level, and each service’s share of quality loss	Weighted aggregation, level bands
M5 Recommendation Engine	Map detected problems to corrective actions	Rule base

3.3 Quality Model

Each dimension is measured by one or two runtime metrics. The metrics were chosen from the golden signals and RED method (Section 2.1) and are available from any service instrumented with a standard Prometheus client library. Table 4 lists the metrics, their sub-weights inside the dimension and their normalisation thresholds.

Table 4: Quality dimensions, metrics and normalisation thresholds

Dimension	Metric (sub-weight)	Good (score 100)	Bad (score 0)
Performance	p95 latency (0.6) mean latency (0.4)	≤ 100 ms ≤ 50 ms	≥ 1000 ms ≥ 500 ms
Reliability	5xx error rate (1.0)	0 %	≥ 5 %
Availability	Share of successful health scrapes (1.0)	100 %	≤ 95 %
Resource efficiency	CPU, % of limit (0.5) Memory, % of limit (0.5)	≤ 50 % ≤ 50 %	≥ 95 % ≥ 95 %
Scalability	Tail ratio p99 / p50 latency (1.0)	≤ 3	≥ 15

Scalability is hard to measure without changing the load deliberately. As a runtime proxy we use the **tail-latency ratio** (p99 divided by median latency): when a service approaches saturation, requests start to queue and the slowest requests grow much faster than the typical request. Throughput is recorded for every service but is not scored, because a low request rate at night is not a quality problem.

3.4 Metric Normalisation

Raw metrics have different units and directions (lower latency is better, higher availability is better). Each metric value x is mapped to a score between 0 and 100 using two thresholds, *good* and *bad*:

$$score(x) = 100 \times clamp((x - bad) / (good - bad), 0, 1)$$

The same formula works for both directions, because the order of *good* and *bad* defines the direction. Fixed thresholds were preferred over min–max scaling across observations, because they give a score the **same meaning in every run** and allow two deployments to be compared. The thresholds in Table 4 are initial values based on common service-level objectives and will be reviewed with the supervisor.

3.5 Weight Assignment

The five dimension weights were derived with AHP. Table 5 shows the pairwise comparison matrix on Saaty’s 1–9 scale; for example, Performance is judged moderately more important than Availability (2) and Scalability (3).

Table 5: AHP pairwise comparison matrix of the quality dimensions

	Performance	Reliability	Availability	Efficiency	Scalability
Performance	1	1	2	2	3
Reliability	1	1	1	2	3
Availability	1/2	1	1	1	2
Efficiency	1/2	1/2	1	1	2
Scalability	1/3	1/3	1/2	1/2	1

The weights are the normalised principal eigenvector of the matrix. The largest eigenvalue is $\lambda_{\max} = 5.055$, giving a consistency index $CI = (\lambda_{\max} - n)/(n - 1) = 0.0139$ and a consistency ratio $CR = CI / RI = 0.012$ ($RI = 1.12$ for $n = 5$). Since CR is well below 0.10, the judgements are consistent. The resulting weights are shown in Figure 2.

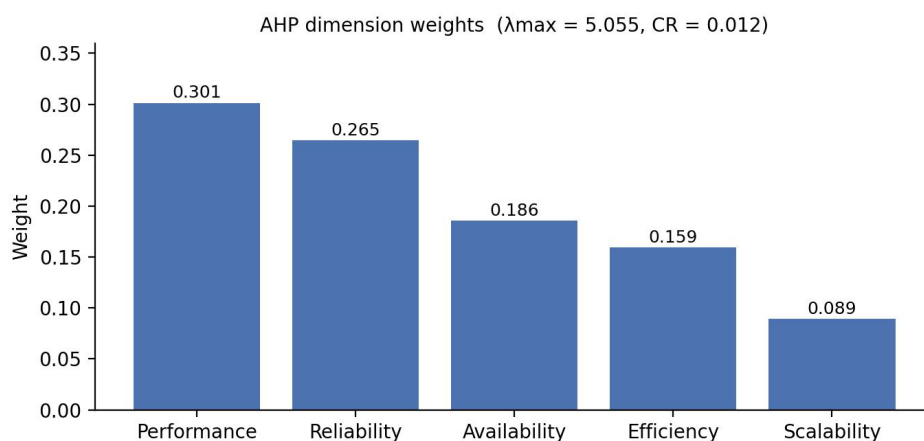


Figure 2: Dimension weights derived by AHP

The framework also implements the **Entropy Weight Method**, which computes weights from the variation of dimension scores across observations, and a **hybrid** weight $w =$

$\alpha \cdot w(AHP) + (1 - \alpha) \cdot w(EWM)$. Comparing the three schemes on the experimental data is planned for the second half (Chapter 6).

3.6 Quality Index and Classification

The Quality Index of a service is the weighted sum of its dimension scores:

$$QI = 0.301 \cdot P + 0.265 \cdot R + 0.186 \cdot A + 0.159 \cdot E + 0.089 \cdot S$$

If a dimension cannot be measured — for example there is no latency when a service receives no traffic — its weight is redistributed over the dimensions that are available, so that missing data does not pull the score towards zero. A service that is **down at the moment of assessment** is handled separately: older samples from before the outage are ignored and only availability is scored, so its QI becomes 0. The QI is then mapped to a quality level (Table 6).

Table 6: Quality levels

QI range	Level	Interpretation
90 – 100	Excellent	All dimensions within target
75 – 89	Good	Minor degradation in one dimension
60 – 74	Fair	Noticeable degradation; investigate
40 – 59	Poor	Serious degradation; action required
0 – 39	Critical	Service failing or unavailable

3.7 System Quality Index and Attribution of Quality Loss

The system QI is the weighted mean of the service QIs, where the weights express business criticality (Order 0.35, Payment 0.30, Product 0.20, User 0.15 in the prototype). To answer the question “which service is responsible?”, the framework computes each service’s share of the total quality loss:

$$share(s) = w(s) \cdot (100 - QI(s)) / \sum_k w(k) \cdot (100 - QI(k))$$

The service with the largest share is reported as the main contributor to degradation.

3.8 Recommendation Engine

The recommendation engine applies simple, explainable rules to the metrics of each service (Table 7). Each rule produces a warning or critical message linked to a quality dimension.

Table 7: Recommendation rules

Condition	Dimension	Recommended action
Availability < 99 %	Availability	Check crash logs, add liveness/readiness probes, run more replicas

Condition	Dimension	Recommended action
Error rate > 1 %	Reliability	Investigate failing dependencies; add circuit breaker, timeouts and retries
p95 latency > 500 ms	Performance	Add caching, optimise slow queries, check downstream latency
CPU > 80 % of limit	Efficiency	Scale out (autoscaling) or raise the CPU limit; profile hot paths
Memory > 80 % of limit	Efficiency	Check for memory leaks or unbounded caches; raise the limit
Tail ratio > 10	Scalability	Requests are queueing: scale out, review pool sizes

4. IMPLEMENTATION

4.1 Technology Stack

Table 8: Technology stack of the prototype

Layer	Technology	Purpose
Microservices	Node.js 22, Express, prom-client	Four demo services with Prometheus instrumentation
Containers	Docker, Docker Compose	One container per service, 0.5 CPU and 256 MB limit each
Metrics	Prometheus 2.53	Scrapes all services every 5 s
Framework	Python 3.11, NumPy, requests, prometheus-client	Quality assessment engine, exporter and CLI
Visualisation	Grafana 11	Dashboard of QI, levels, dimension scores and raw metrics
Load testing	k6 0.52	Low, medium, peak and spike workloads
Testing	pytest	17 unit tests for the framework
Version control	Git, GitHub	Source code, configuration and experiment data

The outline allowed either Spring Boot or Node.js for the services and either Kubernetes or Docker Compose for orchestration. Node.js and Docker Compose were chosen for the first half because they start quickly on a single laptop and keep the prototype small; the framework itself only depends on Prometheus and would work unchanged with services written in any language. A Kubernetes deployment is planned for the second half.

4.2 Microservice Prototype

The prototype is a small e-commerce back end with four services (Figure 1). **User** and **Product** serve catalogue data; **Payment** captures a payment; **Order** is the entry point for checkout and calls the other three services, so a problem in any dependency propagates to it. Each service simulates realistic processing time (for example 20–60 ms for a payment) and exposes a request-latency histogram, a request counter labelled by route and status code, and standard process metrics (CPU time, resident memory).

To create failure scenarios in a controlled way, every service has a **fault-injection endpoint** (/chaos) that can add latency, return HTTP 500 for a given fraction of requests, burn CPU on every request, or allocate and hold memory. The command `qaf chaos payment --error-rate 0.3`, for example, makes 30 % of payment requests fail.

4.3 Monitoring Setup

Prometheus scrapes the four services, the framework’s own exporter and itself. Grafana is provisioned automatically with a Prometheus data source and a dashboard, “Runtime Quality Assessment – Overview”, which shows the system QI, the QI and quality level of each service, the dimension scores, each service’s share of quality loss, and the underlying p95 latency, throughput, error rate, CPU and memory.

4.4 Quality Assessment Engine

The framework is a Python package with one module per framework component (collector, preprocessing, scoring, weighting, recommender, engine, exporter). All thresholds, dimensions, weights, AHP matrix, service criticality weights and quality levels are defined in one YAML configuration file, so the model can be changed without changing the code. The command-line tool provides the following commands:

Table 9: Framework command-line interface

Command	Function
qaf assess	Run one assessment and print dimension scores, QI, level, main contributor and recommendations
qaf serve	Assess every 15 s and publish the results as Prometheus metrics for Grafana
qaf record	Record assessments for an experiment scenario to CSV
qaf summarize	Summarise recorded runs per scenario and service
qaf weights	Show AHP weights and consistency; compute entropy and hybrid weights from recorded data
qaf chaos	Inject or remove faults in a service

4.5 Testing

The framework has 17 automated unit tests. They check normalisation in both directions, handling of missing data, the quality-level bands, AHP weights and detection of inconsistent matrices, entropy weights, outlier removal, that a healthy service is rated Excellent, that a degraded service is identified as the main contributor with the correct recommendations, and that a service which is down scores 0 even when older healthy samples are still in the query window. All tests pass. The services were also tested manually end-to-end, including propagation of a payment failure to the order service.

4.6 Source Code Repository

The complete source code, configuration, experiment scripts and raw results are kept in a Git repository (github.com/Gpalve848/QFA-V1-NOD). The whole environment starts with one

command (`docker compose up`), and all nine experiments can be repeated with one script, which makes the results reproducible.

5. EXPERIMENTAL SETUP AND PRELIMINARY RESULTS

5.1 Experimental Setup

All experiments ran on a single laptop (Intel Core i5-7300HQ, 4 cores, 16 GB RAM, Windows 11) with Docker Desktop. Each microservice container was limited to 0.5 CPU and 256 MB of memory so that saturation can be reached with moderate load. k6 generated traffic with virtual users (VUs) that each send about one request per second with the following mix: 30 % product listing, 20 % product detail, 15 % user lookup and 35 % order checkout (which also calls User, Product and Payment).

For each scenario an automated script (1) restarts all services so that no state is carried over, (2) waits 60 s so that the rate windows are clear, (3) starts the k6 load, (4) injects the fault after 15 s, and (5) after 75 s of warm-up records one assessment every 15 s for 3 minutes (12 assessments per service). **Each scenario was run once**; repeated runs are planned (Chapter 6), so the results below should be read as preliminary.

Table 10: Experiment scenarios

ID	Scenario	Load (k6)	Injected fault
E1	Low	5 VUs	None
E2	Medium	25 VUs	None
E3	Peak	100 VUs	None
E4	Spike	10 → 150 → 10 VUs	None
E5	Payment errors	25 VUs	30 % of payment requests fail
E6	Order latency	25 VUs	+800 ms (± 50 %) on order
E7	Product CPU	25 VUs	30 ms CPU burn per product request
E8	User memory	5 VUs	User service holds 220 MB extra
E9	Payment outage	25 VUs	Payment container stopped

5.2 System Quality Index per Scenario

Figure 3 shows the mean system QI for each scenario, and Table 11 gives the level and the lowest-scoring service. Under low and medium load the system is Excellent. Peak and spike load lower the system QI into the low 90s, and the injected failures lower it further, down to Poor when the payment service is stopped. The ordering of the scenarios matches the expected severity of the conditions.

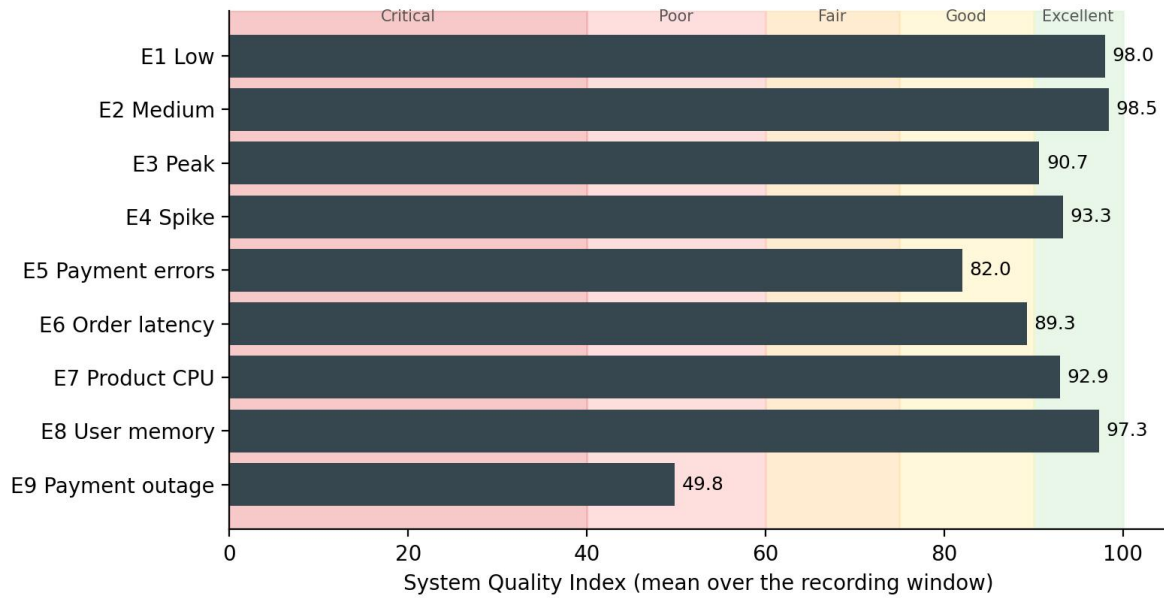


Figure 3: Mean system Quality Index per scenario with quality-level bands

Table 11: System QI and lowest-scoring service per scenario

Scenario	System QI	Level	Lowest service (QI)
E1 Low	98.0	Excellent	Order (94.8)
E2 Medium	98.5	Excellent	Order (96.1)
E3 Peak	90.7	Excellent	Order (74.2)
E4 Spike	93.3	Excellent	Order (81.5)
E5 Payment errors	82.0	Good	Order (71.7)
E6 Order latency	89.3	Good	Order (69.9)
E7 Product CPU	92.9	Excellent	Order (86.1)
E8 User memory	97.3	Excellent	User (91.8)
E9 Payment outage	49.8	Poor	Payment (0.0)

5.3 Service-Level Quality Index

Figure 4 breaks the results down by service. In every scenario the service with the lowest QI is the one where the problem was created or the one that directly suffers from it: the order service under peak and spike load (it calls three other services and is the most CPU-intensive), payment and order in E5, order in E6, product and order in E7, user in E8, and payment and order in E9. Services not involved stay at 99–100.

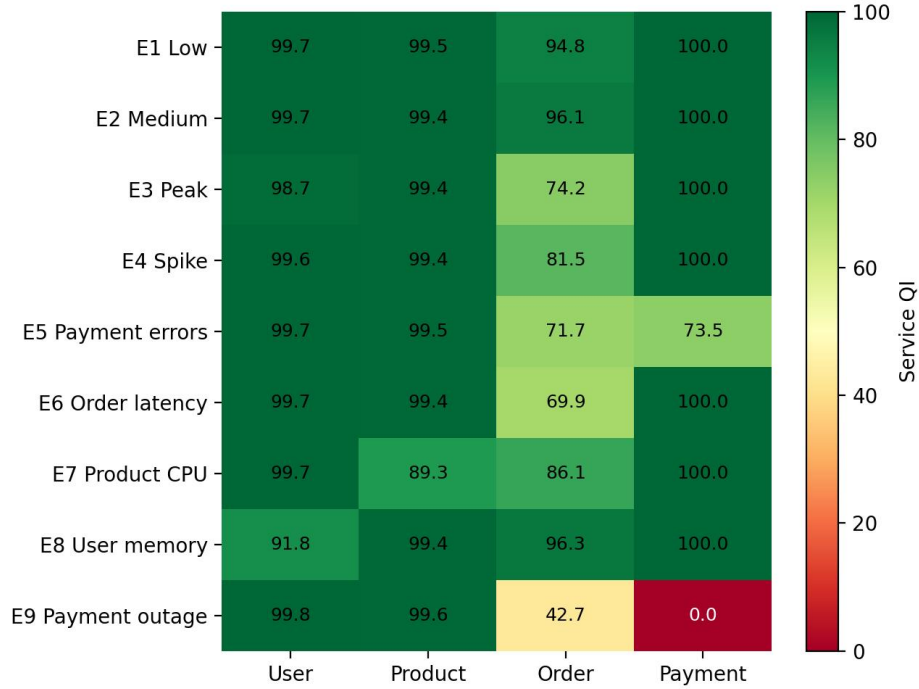


Figure 4: Mean service Quality Index per scenario

5.4 Quality Index over Time

Figure 5 follows the QI through three scenarios. Under peak load (E3), the order service falls gradually from Good to Fair as queues build up, and recovers at the end when k6 starts reducing the load. With 30 % payment errors (E5), both payment and order stay at about 72 (Fair) for the whole window. When payment is stopped (E9), its QI is 0 from the first assessment and order drops to Poor, because most of its checkout requests now fail and time out.

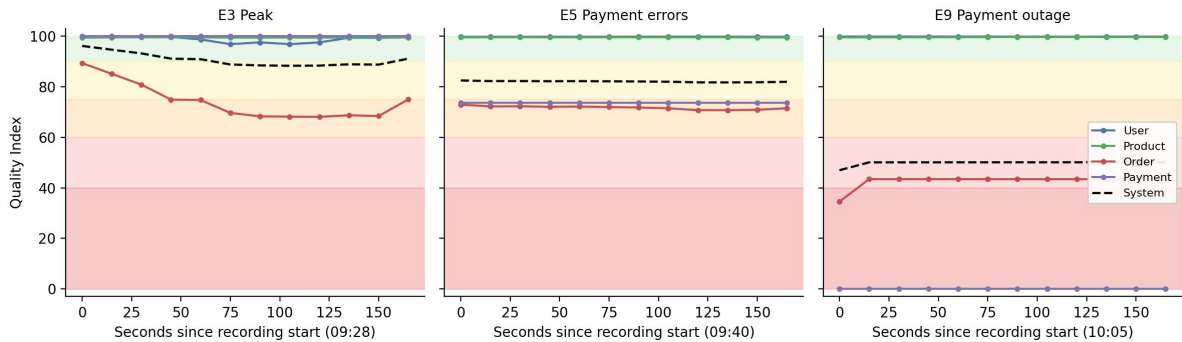


Figure 5: Service and system QI over time for E3, E5 and E9

5.5 Dimension Scores and Recommendations

The dimension scores explain *why* a QI is low (Figure 6). Under peak load, the order service loses mainly on scalability (tail ratio) and performance; with injected latency (E6) its performance score falls to 0 while all other dimensions stay at 100; the CPU fault (E7) shows

up in the efficiency of the product service; and the memory leak (E8) halves the efficiency score of the user service.

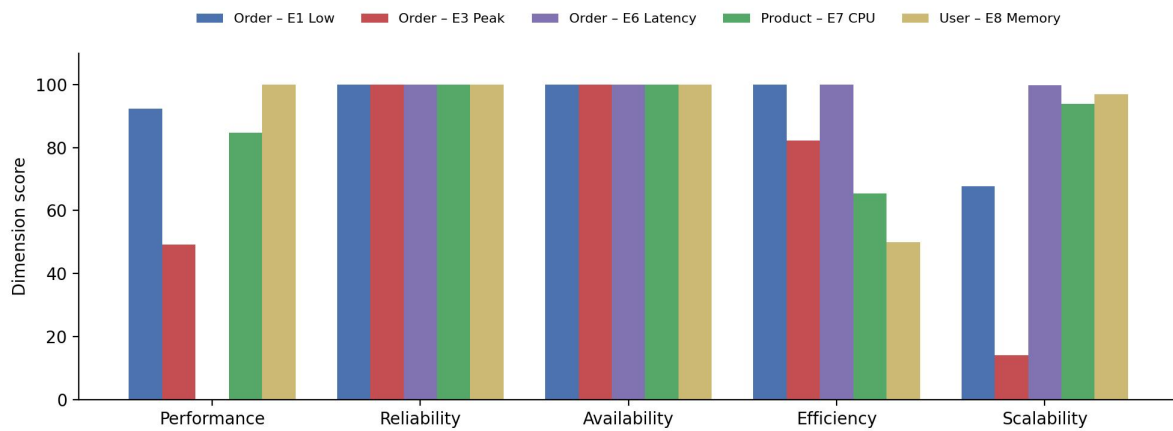


Figure 6: Dimension scores of the affected service in selected scenarios (E6 performance = 0)

The recommendation engine produced the expected advice in each failure scenario: a *critical reliability* message (circuit breaker, timeouts, retries) for order and payment in E5; a *critical performance* message (caching, slow queries) for order in E6; an *efficiency* warning (scale out) for product in E7; a *critical efficiency* message (memory leak) for user in E8; and a *critical availability* message for payment together with critical reliability and performance messages for order in E9.

5.6 Preliminary Comparison with Threshold-Based Monitoring

As a first step towards objective 7, we compared the framework with typical static alert rules used with Prometheus: CPU above 80 % of the limit, memory above 80 %, error rate above 5 %, p95 latency above 1 s, and service down. Table 12 shows which of these rules would fire in each degraded scenario and what the framework reported.

Table 12: Static alert rules compared with the framework (degraded scenarios)

Scenario	Static alerts that fire	Framework output
E3 Peak	None	Order Fair (74.2), p95 696 ms, tail ratio high
E4 Spike	None	Order Good (81.5), p95 561 ms
E5 Payment errors	Error rate (order, payment)	Both Fair; loss split between order and payment
E6 Order latency	Latency (order)	Order Fair (69.9); performance score 0
E7 Product CPU	None	Product 89.3 (CPU 75 %); order latency 515 ms
E8 User memory	Memory (user)	User 91.8 — still Excellent (see 5.7)
E9 Payment outage	Service down; errors and latency (order)	Payment Critical (0.0); order Poor (42.7); system Poor

In three of the seven degraded scenarios (E3, E4 and E7) **no static alert would fire**, because every individual metric stays below its alert threshold, while the QI shows a clear drop and names the affected service. In E7 the root cause (CPU on product at about 75 %) and the visible symptom (latency on order) are on different services; a person reading separate dashboards would have to connect them, while the dimension scores show both. This comparison is preliminary: it uses one run per scenario and one set of alert thresholds. A more structured comparison is planned for the second half.

5.7 Observations and Limitations

The experiments also revealed issues in the framework. Finding them early is one of the main outcomes of the first half:

- **Stale data after an outage (fixed).** In the first run of E9, the stopped payment service still scored 55.8, because the 2-minute query window contained healthy samples from before the outage. The preprocessing module now checks the most recent health sample and scores a service that is currently down on availability only. All results in this report were produced after this fix.
- **State carried between scenarios (fixed).** Memory allocated in E8 was still held by the user service in E9. The experiment script now restarts all services before each scenario.
- **Compensation hides critical conditions (open).** In E8 the user service used 115 % of its memory limit — the container was swapping and would be killed by Kubernetes — yet its QI stayed at 91.8 (Excellent), because the other four dimensions were perfect. Similarly, 27 % errors in E5 only lowered payment to Fair. This is the compensability problem of weighted averages [10]; a non-compensatory rule (for example, capping the QI when any dimension is critical) will be evaluated.
- **Blame is shared along the call chain (open).** In E5 the root cause is the payment service, but the order service, which calls it, shows the same error rate and has a higher business weight, so the framework assigns it a slightly larger share of the loss (order 71.7 vs payment 73.5). Using the service dependency graph to attribute propagated errors to the failing dependency is planned.
- **Tail ratio at low traffic (open).** At very low request rates the p99/p50 ratio is based on few requests and becomes noisy (order scalability 67.7 in E1). A minimum request rate before scoring this dimension will be added.

- **Single runs on one machine.** Each scenario was run once on a laptop, so the numbers include run-to-run noise that has not yet been measured.

6. FUTURE PLAN

Table 13 shows the plan of work from the dissertation outline with the status of each phase at mid-semester.

Table 13: Plan of work and status

Sl. No	Phase	Start – End Date	Work to be done	Status
1	Literature Review & Outline	01.08.2026 – 15.08.2026	Runtime quality attributes, observability literature, problem, gap and outline	COMPLETED
2	Framework & QI Model Design	16.08.2026 – 30.08.2026	Quality dimensions, metrics, normalisation, AHP/entropy weighting, QI and levels	COMPLETED
3	Prototype Development	01.09.2026 – 15.09.2026	Four microservices, Prometheus, Grafana, all framework modules, tests	COMPLETED
4	Experimentation & Validation	15.09.2026 – 04.10.2026	Repeated runs, weighting comparison, comparison with dashboard monitoring	IN PROGRESS (single runs of all 9 scenarios done)
5	Dissertation Review	05.10.2026 – 19.10.2026	Draft to supervisor and additional examiner for review	PENDING
6	Final Compilation & Submission	20.10.2026 – 05.11.2026	Incorporate feedback, final report, demonstration	PENDING

The next steps are:

- (1) **Repeated runs.** Run each scenario at least three times and report the mean and standard deviation of the QI, so that differences between scenarios can be shown to be larger than run-to-run noise.
- (2) **Weighting comparison.** Compute entropy and hybrid weights from the recorded data and compare the rankings of scenarios and services produced by fixed, AHP, entropy and hybrid weights.
- (3) **Non-compensatory rule.** Add and evaluate a rule that caps the QI when any single dimension is critical (Section 5.7), and review the normalisation thresholds with the supervisor.
- (4) **Dependency-aware attribution.** Use the call graph between services so that errors propagated from a failing dependency are attributed to that dependency.
- (5) **Comparison with dashboard monitoring.** Define a structured comparison with a standard Prometheus/Grafana alerting setup: detection of each degradation, time to detect, and correct identification of the responsible service.

- (6) **Kubernetes deployment.** Deploy the prototype on a local Kubernetes cluster with horizontal pod autoscaling to evaluate the scalability dimension under real scaling.
- (7) **Literature.** Extend the literature review with recent peer-reviewed work on QoS-based composite indices, as advised by the supervisor.

7. REFERENCES

- [1] ISO/IEC 25010:2011, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models, International Organization for Standardization, 2011.
- [2] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. New York: McGraw-Hill, 1980.
- [3] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [4] B. Beyer, C. Jones, J. Petoff and N. R. Murphy (eds.), *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016.
- [5] S. Newman, *Building Microservices*, 2nd ed. O’Reilly Media, 2021.
- [6] N. Dragoni et al., “Microservices: Yesterday, Today, and Tomorrow,” in *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [7] B. Gregg, “Thinking Methodically about Performance,” *Communications of the ACM*, vol. 56, no. 2, pp. 45–51, 2013.
- [8] Prometheus Authors, “Prometheus Documentation,” <https://prometheus.io/docs/> (accessed September 2026).
- [9] Grafana Labs, “Grafana k6 Documentation,” <https://grafana.com/docs/k6/> (accessed September 2026).
- [10] OECD and European Commission JRC, *Handbook on Constructing Composite Indicators: Methodology and User Guide*. Paris: OECD Publishing, 2008.
- [11] T. Wilkie, “The RED Method: Key Metrics for Microservices Architecture,” *Grafana Labs blog*, 2018.
- [12] J. Soldani, D. A. Tamburri and W.-J. van den Heuvel, “The Pains and Gains of Microservices: A Systematic Grey Literature Review,” *Journal of Systems and Software*, vol. 146, pp. 215–232, 2018.
- [13] O. Al-Debagy and P. Martinek, “A Metrics Framework for Evaluating Microservices Architecture Designs,” *Journal of Web Engineering*, vol. 19, no. 3–4, pp. 341–370, 2020.
- [14] V. C. Tapia and C. M. Gaona, “Research Opportunities in Microservices Quality Assessment: A Systematic Literature Review,” *Journal of Advances in Information Technology*, vol. 14, no. 5, pp. 991–1002, 2023.
- [15] A. S. Abdelfattah, “Fostering Microservice Maintainability Assurance through a Comprehensive Framework,” in *Proc. IEEE Int. Conf. on Software Maintenance and Evolution (ICSME), Doctoral Symposium*, 2024. arXiv:2407.16873.
- [16] D. Taibi and K. Systä, “A Decomposition and Metric-Based Evaluation Framework for Microservices,” arXiv:1908.08513, 2019.

- [17] Z. Zou, Y. Yun and J. Sun, “Entropy method for determination of weight of evaluating indicators in fuzzy synthetic evaluation for water quality assessment,” *Journal of Environmental Sciences*, vol. 18, no. 5, pp. 1020–1023, 2006, doi: 10.1016/S1001-0742(06)60032-6.
- [18] N. Canbaz Horozlu and T. Serif, “A Multidimensional Decision-Support Framework for Software Quality Assessment in Agile Projects,” *Information*, vol. 17, no. 7, art. 624, 2026, doi: 10.3390/info17070624.

8. ABBREVIATIONS

Table 14: Abbreviations

Abbreviation	Expansion
AHP	Analytic Hierarchy Process
API	Application Programming Interface
CI / CR	Consistency Index / Consistency Ratio
CPU	Central Processing Unit
CSV	Comma-Separated Values
EWM	Entropy Weight Method
HPA	Horizontal Pod Autoscaler
HTTP	Hypertext Transfer Protocol
IQR	Interquartile Range
p50 / p95 / p99	50th / 95th / 99th percentile
PromQL	Prometheus Query Language
QAF	Quality Assessment Framework
QI	Quality Index
QoS	Quality of Service
RED	Rate, Errors, Duration
SRE	Site Reliability Engineering
USE	Utilisation, Saturation, Errors
VU	Virtual User (k6)